

Supplementary ExoFOP Calculations

1. CONTEXT

The TOI table at ExoFOP primarily list planetary, stellar, and system parameters that are provided by the TESS mission. However, there are four values that are calculated by NExSci and added to those values: the predicted mass, predicted radial velocity semi-amplitude, the transmission spectroscopy metric, and the emission spectroscopy metric.

In order to best evaluate which exoplanets discovered by TESS are most suitable for atmospheric characterization, [Kempton et al. \(2018\)](#) introduced two metrics to evaluate exoplanets; the transmission spectroscopy metric (TSM) and emission spectroscopy metric (ESM).

The TSM (given as Eq. 1 in [Kempton et al. 2018](#)) can be defined by the following equation, where R_p is planet radius in Earth radii, M_p is planet masses in Earth masses, $R_\star$ is the stellar radius in solar radii, T_{eq} is the planet's equilibrium temperature in Kelvin, m_J is the apparent stellar magnitude in the J band, and the scale factor is dependent on planetary radius as defined in Table 1 of [Kempton et al. \(2018\)](#)¹:

$$TSM = (scale\ factor) \times \frac{R_p^3 T_{eq}}{M_p R_\star^2} \times 10^{-m_J/5} \quad (1)$$

The ESM (given as Eq. 4 in [Kempton et al. 2018](#)) can be defined by the following equation with additional terms, where $B_{7.5}$ is Planck's function evaluated at 7.5 μm for a given temperature, T_{day} is the dayside temperature of the planet in Kelvin calculated as $1.10 \times T_{eq}$, and m_K is the apparent stellar magnitude in the K band:

$$ESM = 4.29 \times 10^6 \times \frac{B_{7.5}(T_{day})}{B_{7.5}(T_\star)} \times \frac{R_p}{R_\star} \times 10^{-m_K/5} \quad (2)$$

Both equations feature T_{eq} , either explicitly in the case of TSM or implicitly in the case of ESM, where T_{day} is defined as $1.10 \times T_{eq}$. T_{eq} can be defined using the following equation from [Kempton et al. \(2018\)](#) as a function of stellar effective temperature ($T_\star$), stellar radius ($R_\star$), and semi-major axis (a), where the units of $R_\star$ and a must be the same:

$$T_{eq} = T_\star \sqrt{\frac{R_\star}{a}} \left(\frac{1}{4}\right)^{\frac{1}{4}} \quad (3)$$

The semi-major axis, in turn, can be derived from the stellar mass ($M_\star$) and period (P), provided that we assume the mass of the planet is sufficiently smaller than the mass of the star as to be negligible:

$$a^3 = \frac{GM_\star P^2}{4\pi^2} \quad (4)$$

¹ As Table 1 only covers planets up to 10 $R_\oplus$, for any planets larger than that we use the scale factor associated with the bin for 4 to 10 $R_\oplus$ (1.15)

For stars that do not have a stellar mass, but do have a surface gravity ($\log g$) and stellar radius ($R_\star$), we can obtain the stellar mass ($M_\star$) from the following equation:

$$M_\star = \frac{gR_\star^2}{G} \quad (5)$$

As computing the TSM already requires the calculation of a predicted mass in the process, we can also use this predicted mass and the other system properties to get a predicted radial velocity semi-amplitude (K)². This radial velocity semi-amplitude can be expressed as the following equation, which also includes values for planetary orbit inclination (i) and eccentricity (e):

$$K = \left(\frac{2\pi G}{P} \right)^{\frac{1}{3}} \frac{M_p \sin i}{M_\star^{\frac{2}{3}}} \frac{1}{\sqrt{1-e^2}} \quad (6)$$

2. EQUILIBRIUM TEMPERATURE

The equilibrium temperature relies on the stellar mass, and we first check the TESS Follow-up Observing Program (TFOP) SG2/SG4 Working Group spreadsheet and then the Exoplanet Follow-up Observing Program (ExoFOP) website for a stellar mass³. If there is no stellar mass included in either source, we then check for a surface gravity ($\log g$, from ExoFOP) and stellar radius ($R_\star$, from SG2/SG4 then ExoFOP) in order to calculate the stellar mass ($M_\star$) with Equation 5. If these values are present, we use the following Python function which takes surface gravity ($\log g$) in units of cm/s^2 and stellar $R_\star$ in units of solar radii to calculate a stellar mass in units of solar masses:

```
from astropy import units as u
from astropy import coordinates as c
def get_stellar_mass(logg, R_s):
    M_s=(u.cm/(u.s**2)*10**logg)*(R_s*u.Rsun)**2/c.G
    return M_s.to(u.Msun).value
```

Once we have a stellar mass, we calculate the semi-major axis, taking the orbital period (P) in units of days and the stellar mass ($M_\star$) in units of solar masses. We ignore all remaining calculations for TOIs with a period of 9999, which is used by the TFOP SG2/SG4 Working Group as a placeholder for TOIs that don't have periods included in their release. The function for calculating the semi-major axis is as follows, and is an applied form of Equation 4:

```
from astropy import units as u
from astropy import coordinates as c
def get_a(M_s, P):
    a3=c.G*(M_s*u.Msun)*(P*u.d)**2/(4*np.pi**2)
    a=a3**(1/3.)
    return a.decompose()
```

Finally, we calculate the planet candidate's equilibrium temperature as a function of stellar mass ($M_\star$) and stellar radius ($R_\star$) in solar units, orbital period (P) in days, and stellar effective temperature ($T_\star$) in Kelvin, based on Equation 3:

² https://sites.astro.caltech.edu/~srk/BlackHoles/Literature/RV_Derivation.pdf

³ ExoFOP uses this approach for both candidates and confirmed planets; note that the published parameters at the Exoplanet Archive may be more reliable for confirmed planets

```

from astropy import units as u
def get_Teq(M_s, P, R_s, Teff):
    a=get_a(M_s, P)
    R_s_a=(R_s*u.R_sun/a).decompose()
    Teq=Teff*R_s_a**0.5/4**0.25
    return Teq.value

```

3. TRANSMISSION SPECTROSCOPY METRIC (TSM) AND PLANET MASS

Each time we carry out the calculations, we refresh the input values from the TFOP SG2/SG4 Working Group spreadsheet. If they are available, we use the R_p and $R_\star$ from that spreadsheet. If they are not available there, we use the values from the ExoFOP TOI table⁴ if available. For m_J , we only use the value from the TFOP SG2/SG4 Working Group spreadsheet, and further calculations are not carried out if this value is missing. T_{eq} is derived from the calculations discussed in Section 2.

The TSM scale factor is calculated using the values from the table in Kempton et al. (2018) in the following Python function:

```

def get_TSM_scale_factor(radius):
    if radius <= 1.5:
        return 0.19
    elif radius <= 2.75:
        return 1.26
    elif radius <= 4:
        return 1.28
    else:
        return 1.15

```

The planet mass is approximated following the recommendation of Kempton et al. (2018) to use the mass-radius relationship from Chen and Kipping (2017) as implemented by Louie et al. (2018). We further extend the equation from Louie et al. (2018) to the Jovian regime by setting the planet mass to $317 M_{\text{Jup}}$ for planets larger than $14.26 R_\oplus$, as that paper only addressed terrestrial and Neptunian planets. This is done using the following function:

```

def predict_mass(radius):
    if radius > 14.26:
        return 317
    if radius < 1.23:
        return 0.9718*radius**3.58
    else:
        return 1.436*radius**1.70

```

The TSM value is then calculated using a function that includes both of these calculations.

```

def get_TSM_value(radius, Teq, star_radius, Jmag):
    return get_TSM_scale_factor(radius)*Teq*radius**3/ \
        (predict_mass(radius)*star_radius**2)*10**(-Jmag/5)

```

⁴ https://exofop.ipac.caltech.edu/tess/view_toi.php

4. EMISSION SPECTROSCOPY METRIC (ESM)

With the same motivations as calculating the TSM, we pull values from the TFOP SG2/SG4 Working Group spreadsheet before carrying out any calculations. If they are available, we use the R_p , $R_\star$, and $T_\star$ from that spreadsheet as previously defined. If they are not available in that spreadsheet, then the values from the ExoFOP TOI table are used⁵. For m_K , we only use the value from the SG2/SG4 TFOP Working Group spreadsheet. T_{eq} is derived from the calculations discussed in Section 2. If any of these values are missing, we do not calculate an ESM value.

As the necessary equation involves modeling Planck’s function at a given wavelength ($7.5 \mu\text{m}$) and temperature, we use astropy’s inbuilt function for modeling a black-body to carry out this part of the calculation. We also use astropy’s units module in order to properly handle the black-body modelling.

We currently also calculate a conversion factor based on the true units for the planet radius and stellar radius being in Earth radii and solar radii, respectively, and rely on astropy’s units module for this. This allows us to ensure that our final ESM value is unitless.

```
from astropy.modeling.models import BlackBody
from astropy import units as u
conversion=(1*u.earthRad/(1*u.solRad)).decompose().value**2
def get_ESM_value( $T_{eq}$ ,  $T_{eff}$ ,  $R_p$ ,  $R_s$ ,  $K_{mag}$ ):
    depth = 10**6 * conversion * ( $R_p/R_s$ )**2
    ESM_bb_Tday=BlackBody(temperature=1.1* $T_{eq}$ *u.K)
    ESM_bb_Teff=BlackBody(temperature= $T_{eff}$ *u.K)
    wavelength=7.5*u.micron
    part1=(ESM_bb_Tday(wavelength)/ESM_bb_Teff(wavelength)).value
    part2=(depth)*10**(- $K_{mag}/5$ )
    return 4.29*part1*part2
```

5. PREDICTED RADIAL VELOCITY SEMI-AMPLITUDE

Using the predicted planet mass calculated in Section 3, the stellar mass from Section 2, and the period from SG2/SG4, the Predicted RV Semi-amplitude can be calculated using Equation 6. This equation does also include planetary inclination and eccentricity, but for our purposes we presume that orbits are circular and edge-on.

```
def get_K( $P$ ,  $m_1$ ,  $m_2$ ,  $i=90$ ,  $e=0$ ):
    part1=((2*np.pi*c.G)/( $P$ *u.d))**(1/3)
    part2=( ( $m_2$ *u.Mearth)/ ( $m_1$ *u.Msun))**(2/3))*np.sin( $i$ *u.degree)
    part3=(1- $e$ **2)**(-0.5)
    total=(part1*part2*part3).to(u.m/u.s)
    return total.value
```

6. WORKED EXAMPLE

⁵ https://exofop.ipac.caltech.edu/tess/view_toi.php

The following table provides an example of the ESM and TSM calculations for a single TOI, TOI-664.01. We list the provenance of each value, and for those that we calculate we list the Python function that is used to calculate that parameter.

TOI-664.01			
Parameter	Value	Units	Provenance
$M_{\star}$	1.516	$M_{\odot}$	<code>get_stellar_mass</code>
$\log g$	3.72765	cm/s^2	ExoFOP
$R_{\star}$	2.79	$R_{\odot}$	SG2/SG4
P	4.736	days	SG2/SG4
a	0.0634	AU	<code>get_a</code>
$T_{\star}$	5302	K	SG2/SG4
T_{eq}	1699	K	<code>get_Teq</code>
R_p	14.03	$R_{\oplus}$	SG2/SG4
m_J	6.616	mag	SG2/SG4
m_K	6.122	mag	SG2/SG4
M_p	128.0	$M_{\oplus}$	<code>predict_mass</code>
K	36.9	m/s	<code>get_K</code>
TSM scale	1.15		<code>get_TSM_scale_factor</code>
TSM value	257		<code>get_TSM_value</code>
ESM value	132		<code>get_ESM_value</code>

7. SOFTWARE

Software: `astropy` (Astropy Collaboration et al. 2013, 2018, 2022), `numpy` (Harris et al. 2020), `pandas` (McKinney 2010)

REFERENCES

- Astropy Collaboration, Price-Whelan, A. M., Lim, P. L., et al. (2022). The Astropy Project: Sustaining and Growing a Community-oriented Open-source Project and the Latest Major Release (v5.0) of the Core Package. *ApJ*, 935(2):167.
- Astropy Collaboration, Price-Whelan, A. M., Sipőcz, B. M., et al. (2018). The Astropy Project: Building an Open-science Project and Status of the v2.0 Core Package. *AJ*, 156(3):123.
- Astropy Collaboration, Robitaille, T. P., Tollerud, E. J., et al. (2013). Astropy: A community Python package for astronomy. *A&A*, 558:A33.
- Chen, J. and Kipping, D. (2017). Probabilistic Forecasting of the Masses and Radii of Other Worlds. *ApJ*, 834(1):17.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.
- Kempton, E. M. R., Bean, J. L., Louie, D. R., et al. (2018). A Framework for Prioritizing the TESS Planetary Candidates Most Amenable to Atmospheric Characterization. *PASP*, 130(993):114401.

- Louie, D. R., Deming, D., Albert, L., Bouma, L. G., Bean, J., and Lopez-Morales, M. (2018). Simulated JWST/NIRISS Transit Spectroscopy of Anticipated Tess Planets Compared to Select Discoveries from Space-based and Ground-based Surveys. *PASP*, 130(986):044401.
- McKinney, W. (2010). Data structures for statistical computing in python. In van der Walt, S. and Millman, J., editors, *Proceedings of the 9th Python in Science Conference*, pages 56 – 61.